

ORBITAL Development Roadmap

300-Hour Execution Strategy

ORBITAL is a C++-based autonomous asteroid mining mission operating system designed to simulate robotic mining operations near Mars and the asteroid belt. This roadmap structures the project into manageable engineering phases optimized for hackathon viability, advanced C++ learning, and long-term scalability.

Overall Execution Model

Build a reusable simulation platform rather than a one-off application. Every phase must produce visible progress, runnable software, and reusable architecture.

Phase 1 — Foundation Setup (35 Hours)

- Configure C++20 project with CMake
- Integrate OpenGL, GLFW, Dear ImGui, EnTT, Eigen, spdlog
- Create folder structure and logging system
- Establish debug/release builds

Deliverable:

A working ORBITAL application window with rendering and GUI support.

Phase 2 — Rendering + GUI (35 Hours)

- Build mission-control-style interface
- Create telemetry panels and viewport systems
- Add simulation controls and debug overlays
- Design a professional aerospace dashboard aesthetic

Deliverable:

A functional mission-control GUI.

Phase 3 — ECS Architecture (40 Hours)

- Implement Entity Component System architecture
- Create entities for rovers, asteroids, and stations
- Add movement, mining, battery, and telemetry systems

- Optimize runtime structure for scalability

Deliverable:

A real simulation engine core with autonomous entity updates.

Phase 4 — .ROX Parser (45 Hours)

- Build lexer and parser for `.rox` mission files
- Implement syntax validation and mission loading
- Generate ECS entities dynamically from mission data
- Add human-readable parser error reporting

Deliverable:

A fully operational data-driven mission scripting system.

Phase 5 — Simulation Systems (55 Hours)

- Implement rover navigation and mining logic
- Create asteroid depletion and battery systems
- Build telemetry update pipelines
- Add mission progression logic

Deliverable:

Visually convincing autonomous mining operations.

Phase 6 — Telemetry + Replay (30 Hours)

- Create replay snapshot architecture
- Implement mission export and playback systems
- Add timeline scrubbing and telemetry history

Deliverable:

Professional mission replay and analysis capability.

Phase 7 — Polish + Optimization (35 Hours)

- Optimize ECS performance and rendering
- Fix stability issues and parser edge cases
- Improve visual consistency and telemetry feedback
- Profile runtime performance

Deliverable:
Stable and presentation-ready software.

Phase 8 — Presentation + Packaging (25 Hours)

- Build final demo mission scenario
- Create screenshots and architecture diagrams
- Prepare README and pitch materials
- Rehearse live demo flow

Deliverable:
Hackathon-ready final build and presentation assets.

Critical Engineering Rules

1. Do not add multiplayer, networking, or cloud systems.
 2. Keep rendering in 2D during MVP development.
 3. Avoid premature optimization and feature creep.
 4. Keep the engine runnable after every session.
 5. Focus on architecture quality over visual effects.
-

Final Vision

ORBITAL is intended to become a foundation for future autonomous astrophysics and asteroid mining infrastructure. The long-term objective is to evolve the platform into a scalable robotics mission operating system capable of supporting advanced rover coordination, orbital mission planning, and deep-space industrial automation.

Project Timeline Summary

Phase	Focus	Hours
1	Foundation Setup	35
2	Rendering + GUI	35
3	ECS Architecture	40
4	.ROX Parser	45
5	Simulation Systems	55
6	Telemetry + Replay	30

7	Polish + Optimization	35
8	Presentation + Packaging	25

Success is defined not by the number of features added, but by the creation of a believable, extensible aerospace simulation platform with strong engineering architecture, advanced C++ systems design, and a stable, compelling live demonstration.